

MASTER INTELLIGENCE ARTIFICIELLE
TRAVAUX DIRIGÉS — MACHINE LEARNING

TD N° 07

Naive Bayes & Raisonnement Bayésien

Durée : 2 heures	Niveau : Intermédiaire
Datasets : Météo (cours) + BankChurners + SMS Spam	Outils : sklearn, numpy, pandas, matplotlib

Objectifs du TD

- Appliquer le **théorème de Bayes** manuellement sur l'exemple météo du cours
- Comprendre et vérifier le **lissage de Laplace** pour éviter les probabilités nulles
- Comparer **GaussianNB**, **BernoulliNB** et **MultinomialNB** — savoir lequel choisir selon les données
- Analyser l'impact de la **violation de l'hypothèse d'indépendance** sur les performances
- Appliquer Naive Bayes à un problème réel de **détection de spam SMS**
- Appliquer **GaussianNB sur BankChurners** et interpréter les probabilités a posteriori
- **Comparer** Naive Bayes avec la Régression Logistique — modèles génératifs vs discriminatifs

Pr. BEN LAHMAR EL Habib · Semestre 2024–2025

Contexte et Rappels Théoriques

Naive Bayes est souvent le premier algorithme probabiliste présenté en ML : simple, rapide, et étonnamment efficace malgré son hypothèse forte d'indépendance. Comprendre ses fondements revient à maîtriser le **raisonnement probabiliste** — une compétence transversale à tout le ML.

Théorème de Bayes — Fondement de l'algorithme

$$P(C|X) = P(X|C) \cdot P(C) / P(X) \quad \text{avec } P(X) = \sum_c P(X|c) \cdot P(c)$$

(normalisation)

Rappel de cours — Naive Bayes — Hypothèse d'indépendance conditionnelle

- **Hypothèse naïve** : les features sont conditionnellement indépendantes étant donné la classe

$$\text{Donc : } P(X|C) = P(x_1|C) \cdot P(x_2|C) \cdot \dots \cdot P(x_n|C) = \prod_i P(x_i|C)$$

- **Décision** : $\hat{y} = \operatorname{argmax}_c P(C=c) \cdot \prod_i P(x_i|C=c)$ — on choisit la classe de probabilité maximale
- **En pratique** : on utilise les log-probabilités pour éviter l'underflow numérique
- **Surprise** : l'hypothèse est presque toujours violée, mais NB reste compétitif !

Rappel de cours — Les 3 variantes de Naive Bayes

Variante	Modèle $P(x_i C)$	Données adaptées	Cas d'usage
GaussianNB	Gaussienne $N(\mu, \sigma^2)$	Features continues	Iris, churn, médical
BernoulliNB	Bernoulli (0/1)	Features binaires	Présence/absence de mots
MultinomialNB	Multinomiale (comptages)	Features entières ≥ 0	Fréquences de mots (TF)

- **GaussianNB** : estime μ_c et σ_c pour chaque feature par classe $\rightarrow$ probabilité via la densité gaussienne
- **BernoulliNB** : $P(x_i=1|C) = \theta_{i,c}$ — avec lissage de Laplace pour éviter les $P=0$
- **MultinomialNB** : $P(x_i|C) = (N_{i,c} + \alpha) / (N_c + \alpha \cdot |V|)$ — $\alpha=1$: lissage de Laplace

Rappel de cours — Lissage de Laplace — Éviter les Probabilités Nulles

$$\text{Formule avec lissage : } P(x_i|C) = (N(x_i, C) + \alpha) / (N(C) + \alpha \cdot |V|)$$

- **Problème sans lissage** : si un mot n'apparaît jamais dans la classe +1, $P(\text{mot}|+1)=0 \rightarrow$ tout le produit devient 0, peu importe les autres features
- $\alpha=0$: pas de lissage (dangereux), $\alpha=1$: lissage de Laplace (défaut), $\alpha<1$: lissage partiel
- $|V|$ = nombre de valeurs possibles (taille du vocabulaire pour le texte)
- **sklearn** : paramètre **var_smoothing** pour GaussianNB, **alpha** pour Bernoulli et MultinomialNB

Préparez l'environnement :

```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.naive_bayes import GaussianNB, BernoulliNB, MultinomialNB
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import (train_test_split, cross_val_score,
                                     StratifiedKFold)
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, roc_auc_score,
                             f1_score)
from sklearn.pipeline import Pipeline
import warnings ; warnings.filterwarnings('ignore')

# Dataset BankChurners (encodage standard)
raw = pd.read_csv('BankChurners.csv')
data = raw[raw.columns[:-2]].copy()
data['churn'] = (data['Attrition_Flag']=='Attrited Customer').astype(int)
data['Gender'] = data['Gender'].map({'M':1, 'F':0})
data['Marital_Status'] =
data['Marital_Status'].map({'Single':0, 'Married':1, 'Divorced':2, 'Unknown':-1})
edu_map={'Unknown':0, 'Uneducated':1, 'High School':2, 'College':3, 'Graduate':4, 'Post-
Graduate':5, 'Doctorate':6}
inc_map={'Unknown':0, 'Less than $40K':1, '$40K - $60K':2, '$60K - $80K':3, '$80K -
$120K':4, '$120K +':5}
data['Education_Level'] = data['Education_Level'].map(edu_map)
data['Income_Category'] = data['Income_Category'].map(inc_map)
data = pd.get_dummies(data, columns=['Card_Category'], prefix='card',
drop_first=True, dtype=int)
feat_cols = [c for c in data.columns if c not in
['Attrition_Flag', 'churn', 'CLIENTNUM']]
X_b = data[feat_cols] ; y_b = data['churn']
X_tr_b, X_te_b, y_tr_b, y_te_b = train_test_split(X_b, y_b, test_size=0.2,
random_state=42, stratify=y_b)
```

Partie 1 — Calcul Manuel avec le Théorème de Bayes

Durée estimée : 35 minutes | Reprendre l'exemple météo du cours pour en maîtriser chaque étape de calcul.

Q 1.1

Reprise de l'exemple météo — calcul from scratch

Consigne

À partir du dataset météo du cours (14 jours de données), calculez manuellement les probabilités a priori et les vraisemblances, puis prédiriez pour un nouveau jour.

Le dataset météo utilisé en cours :

Jour	Ciel	Température	Humidité	Vent	Jouer ?
1	Ensoleillé	Chaud	Élevée	Non	Non
2	Ensoleillé	Chaud	Élevée	Oui	Non
3	Nuageux	Chaud	Élevée	Non	Oui
4	Pluie	Doux	Élevée	Non	Oui
5	Pluie	Frais	Normale	Non	Oui
6	Pluie	Frais	Normale	Oui	Non
7	Nuageux	Frais	Normale	Oui	Oui
8	Ensoleillé	Doux	Élevée	Non	Non
9	Ensoleillé	Frais	Normale	Non	Oui
10	Pluie	Doux	Normale	Non	Oui
11	Ensoleillé	Doux	Normale	Oui	Oui
12	Nuageux	Doux	Élevée	Oui	Oui
13	Nuageux	Chaud	Normale	Non	Oui
14	Pluie	Doux	Élevée	Oui	Non

```
# Reproduction du dataset météo
meteo = pd.DataFrame({
    'Ciel': ['Ensoleillé', 'Ensoleillé', 'Nuageux', 'Pluie', 'Pluie',
            'Pluie', 'Nuageux', 'Ensoleillé', 'Ensoleillé', 'Pluie',
            'Ensoleillé', 'Nuageux', 'Nuageux', 'Pluie'],
    'Temperature': ['Chaud', 'Chaud', 'Chaud', 'Doux', 'Frais',
                   'Frais', 'Frais', 'Doux', 'Frais', 'Doux',
                   'Doux', 'Doux', 'Chaud', 'Doux'],
    'Humidite': ['Élevée', 'Élevée', 'Élevée', 'Élevée', 'Normale',
                'Normale', 'Normale', 'Élevée', 'Normale', 'Normale',
                'Normale', 'Élevée', 'Normale', 'Élevée'],
    'Vent': ['Non', 'Oui', 'Non', 'Non', 'Non',
            'Oui', 'Oui', 'Non', 'Non', 'Non',
            'Oui', 'Oui', 'Non', 'Oui']})
```

```

    'Jouer':      ['Non','Non','Oui','Oui','Oui',
                  'Non','Oui','Non','Oui','Oui',
                  'Oui','Oui','Oui','Non'],
    })

# — ÉTAPE 1 : Probabilités a priori —————
n_total = len(meteo)
n_oui    = (meteo['Jouer']=='Oui').sum()
n_non    = (meteo['Jouer']=='Non').sum()

P_oui = n_oui / n_total
P_non = n_non / n_total
print(f'P(Jouer=Oui) = {n_oui}/{n_total} = {P_oui:.4f}')
print(f'P(Jouer=Non) = {n_non}/{n_total} = {P_non:.4f}')

# — ÉTAPE 2 : Vraisemblances P(feature|classe) —————
features = ['Ciel','Temperature','Humidite','Vent']

print('\n=== TABLE DE VRAISEMBLANCE ===')
for feat in features:
    print(f'\n-- {feat} --')
    for val in meteo[feat].unique():
        for cls in ['Oui','Non']:
            subset = meteo[meteo['Jouer']==cls]
            n_cls = len(subset)
            n_val_cls = (subset[feat]==val).sum()
            p = n_val_cls / n_cls
            print(f'  P({feat}={val:12s} | Jouer={cls}) = {n_val_cls}/{n_cls} =
{p:.4f}')
```

```

# — ÉTAPE 3 : Prédiction d'un nouveau jour —————
# Nouveau jour : Ensoleillé, Frais, Normale, Oui
nouveau_jour = {'Ciel':'Ensoleillé','Temperature':'Frais',
                  'Humidite':'Normale','Vent':'Oui'}

def bayes_predict(data, target_col, new_obs):
    """Naive Bayes from scratch - sans lissage de Laplace."""
    classes = data[target_col].unique()
    n = len(data)
    scores = {}

    for cls in classes:
        subset = data[data[target_col]==cls]
        # Probabilité a priori
        p_cls = len(subset) / n
        log_prob = np.log(p_cls) # log pour éviter underflow

        # Vraisemblances
        for feat, val in new_obs.items():
            n_val_cls = (subset[feat]==val).sum()
            p_feat = n_val_cls / len(subset)
            if p_feat == 0:
                log_prob = -np.inf # Problème sans lissage !
                break
            log_prob += np.log(p_feat)
```

```

    scores[cls] = log_prob

    return max(scores, key=scores.get), scores

pred, scores = bayes_predict(meteo, 'Jouer', nouveau_jour)
print(f'Nouveau jour : {nouveau_jour}')
print(f'Log-scores    : {scores}')
print(f'Prédiction    : Jouer = {pred}')

```

Questions d'analyse

1. Vérifiez manuellement le calcul pour $P(\text{Jouer}=\text{Oui} \mid \text{Ensoleillé}, \text{Frais}, \text{Normale}, \text{Vent}=\text{Oui})$: $P(\text{Oui}) \times P(\text{Ensoleillé} \mid \text{Oui}) \times P(\text{Frais} \mid \text{Oui}) \times P(\text{Normale} \mid \text{Oui}) \times P(\text{Vent}=\text{Oui} \mid \text{Oui}) = ?$
2. Comparez avec $P(\text{Jouer}=\text{Non} \mid \text{Ensoleillé}, \text{Frais}, \text{Normale}, \text{Vent}=\text{Oui})$. Quelle classe obtient le score le plus élevé ?
3. Maintenant testez avec un jour **Nuageux, Chaud, Élevée, Oui**. Y a-t-il un problème de probabilité nulle ? Dans quelle vraisemblance ?

Q 1.2

Lissage de Laplace — résoudre le problème des probabilités nulles

Consigne

Implémentez le lissage de Laplace dans votre fonction Naive Bayes from scratch. Comparez les prédictions avec et sans lissage.

```

def bayes_predict_laplace(data, target_col, new_obs, alpha=1):
    """Naive Bayes with Laplace smoothing."""
    classes = data[target_col].unique()
    n = len(data)
    scores = {}

    for cls in classes:
        subset = data[data[target_col]==cls]
        p_cls = len(subset) / n
        log_prob = np.log(p_cls)

        for feat, val in new_obs.items():
            # Nombre de valeurs uniques possibles pour cette feature
            n_values = data[feat].nunique()
            n_val_cls = (subset[feat]==val).sum()
            # Lissage : (N(x,C) + alpha) / (N(C) + alpha * |V|)
            p_feat = (n_val_cls + alpha) / (len(subset) + alpha * n_values)
            log_prob += np.log(p_feat)

        scores[cls] = log_prob

    return max(scores, key=scores.get), scores

```

```
# Cas problématique : Nuageux, Chaud, Élevée, Oui
cas_probleme = {'Ciel':'Nuageux','Temperature':'Chaud',
                'Humidite':'Élevée','Vent':'Oui'}

print('=== SANS LISSAGE ===')
pred_sl, scores_sl = bayes_predict(meteo, 'Jouer', cas_probleme)
print(f'Scores: {scores_sl} → Préd: {pred_sl}')

print('\n=== AVEC LISSAGE DE LAPLACE (α=1) ===')
pred_l, scores_l = bayes_predict_laplace(meteo, 'Jouer', cas_probleme)
print(f'Scores: {scores_l} → Préd: {pred_l}')

print('\n=== COMPARAISON α=0.1, α=1, α=5 ===')
for alpha in [0.1, 1, 5]:
    pred, scores = bayes_predict_laplace(meteo, 'Jouer', cas_probleme, alpha=alpha)
    print(f'  α={alpha:.1f} : Oui={scores["Oui"]:.4f} Non={scores["Non"]:.4f} → {pred}')
```

🔍 Indice

- Pour calculer $P(\text{Nuageux, Chaud, Élevée, Vent=Oui} \mid \text{Non})$: cherchez combien de fois chaque combinaison apparaît dans les jours Non
- Le problème survient si une combinaison **n'existe pas** dans les données d'entraînement
- Avec Laplace $\alpha=1$: même si $N(x,C)=0$, la probabilité devient $\alpha/(N(C)+\alpha \cdot |V|) > 0$

Questions d'analyse

4. Sans lissage, le calcul produit-il $-\infty$? Quelle vraisemblance est la cause du problème ?
5. Avec $\alpha=1$ (Laplace), la prédiction change-t-elle par rapport au cas sans problème de zéro ? Quel impact le lissage a-t-il sur les probabilités non-nulles ?
6. Que se passe-t-il si α est très grand (ex : $\alpha=100$) ? Les prédictions convergent vers quoi ?

Partie 2 — Les Trois Variantes Sklearn

Durée estimée : 25 minutes | Choisir le bon Naive Bayes selon la nature des features.

Q 2.1

GaussianNB sur BankChurners — features continues

Consigne

Appliquez GaussianNB sur BankChurners. Inspectez les paramètres appris (moyennes et variances par classe) et évaluez les probabilités a posteriori.

```
# GaussianNB — adapté aux features numériques continues
gnb = GaussianNB()
gnb.fit(X_tr_b, y_tr_b)
y_pred_gnb = gnb.predict(X_te_b)
y_prob_gnb = gnb.predict_proba(X_te_b)[:, 1]

print('=== GaussianNB — BankChurners ===')
print(classification_report(y_te_b, y_pred_gnb,
                             target_names=['Existing (0)', 'Attrited (1)']))
print(f'AUC-ROC : {roc_auc_score(y_te_b, y_prob_gnb):.4f}')

# Paramètres appris : moyennes et variances par classe
print('\n=== PARAMÈTRES APPRIS PAR GAUSSIANNB ===')
print(f'Classes : {gnb.classes_}')
print(f'Probabilités a priori : {gnb.class_prior_.round(4)}')

# Comparer les moyennes des top features par classe
means_df = pd.DataFrame(gnb.theta_,
                        index=['Existing', 'Attrited'],
                        columns=X_tr_b.columns)

top_feats = ['Total_Trans_Ct', 'Total_Revolving_Bal',
             'Avg_Utilization_Ratio', 'Months_Inactive_12_mon',
             'Total_Trans_Amt', 'Contacts_Count_12_mon']

print('\nMoyennes des top features par classe :')
print(means_df[top_feats].round(2).to_string())

# Visualisation des distributions par classe pour 2 features clés
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, feat in zip(axes, ['Total_Trans_Ct', 'Avg_Utilization_Ratio']):
    for cls, label, color in [(0, 'Existing', '#1A7A6E'), (1, 'Attrited', '#9C3D38')]:
        mask = y_tr_b == cls
        ax.hist(X_tr_b.loc[mask, feat], bins=30, alpha=.5,
                color=color, label=label, density=True)
    # Superposer la gaussienne apprise
    mu = gnb.theta_[cls, list(X_tr_b.columns).index(feat)]
    sigma = np.sqrt(gnb.var_[cls, list(X_tr_b.columns).index(feat)])
    x_range = np.linspace(X_tr_b[feat].min(), X_tr_b[feat].max(), 100)
    ax.plot(x_range,
            (1/(sigma*np.sqrt(2*np.pi)))*np.exp(-0.5*((x_range-mu)/sigma)**2),
            color=color, lw=2.5, ls='--')
```

```
ax.set_title(f'Distribution de {feat} - GaussianNB', fontweight='bold')
ax.legend()
plt.tight_layout() ; plt.show()
```

📖 Ce que GaussianNB apprend

- Pour chaque feature i et chaque classe c , GaussianNB estime μ_{ic} (moyenne) et σ^2_{ic} (variance)
- La vraisemblance est : $P(x_i|C=c) = (1/\sqrt{2\pi\sigma^2_{ic}}) \times \exp(-(x_i-\mu_{ic})^2 / 2\sigma^2_{ic})$
- Le **var_smoothing** (1e-9 par défaut) ajoute une petite valeur à σ^2 pour éviter la division par zéro
- GaussianNB suppose que chaque feature suit une **loi normale** par classe — hypothèse souvent grossière en pratique

Questions d'analyse

7. En regardant les moyennes par classe, quelle feature a la plus grande différence de moyenne entre Existing et Attrited ? Est-ce cohérent avec votre EDA (TD3) ?
8. Les histogrammes superposés avec les gaussiennes apprisent-ils bien ajustés ? Y a-t-il des features dont la distribution n'est pas gaussienne ?
9. L'AUC-ROC de GaussianNB est-il comparable à celui de la Régression Logistique (TD4) ou du SVM (TD6) ? Qu'est-ce que cela révèle sur l'hypothèse gaussienne pour ce dataset ?

Q 2.2

BernoulliNB — features binaires

Consigne

Créez des features binaires à partir de BankChurners (seuillage) et appliquez BernoulliNB. Comparez avec GaussianNB.

```
# Créer des features binaires par seuillage
X_bin = pd.DataFrame()

# Seuillage sur les features numériques (médiane comme seuil)
num_feats_bin = ['Total_Trans_Ct', 'Total_Revolving_Bal',
                  'Avg_Utilization_Ratio', 'Months_Inactive_12_mon',
                  'Credit_Limit', 'Contacts_Count_12_mon']

for feat in num_feats_bin:
    median_val = X_b[feat].median()
    X_bin[f'{feat}_above_median'] = (X_b[feat] > median_val).astype(int)

# Ajouter les features déjà binaires
binary_feats = ['Gender'] + [c for c in X_b.columns if c.startswith('card_')]
for feat in binary_feats:
    X_bin[feat] = X_b[feat].astype(int)

print(f'Features binaires créées : {X_bin.shape[1]}')
print(X_bin.head(3))
```

```

X_bin_tr, X_bin_te = X_bin.loc[X_tr_b.index], X_bin.loc[X_te_b.index]

# BernoulliNB
bnb = BernoulliNB(alpha=1.0)
bnb.fit(X_bin_tr, y_tr_b)
y_pred_bnb = bnb.predict(X_bin_te)

print('\n=== BernoulliNB - Features Binaires ===')
print(classification_report(y_te_b, y_pred_bnb,
                             target_names=['Existing', 'Attrited']))

# Comparer les probabilités log P(feature=1|class)
feat_proba = pd.DataFrame(
    np.exp(bnb.feature_log_prob_),
    index=['Existing', 'Attrited'],
    columns=X_bin_tr.columns
).T
feat_proba['Diff (Attrited-Existing)'] = feat_proba['Attrited'] -
feat_proba['Existing']
print('\nProbabilités P(feature=1|classe) - top discriminantes :')
print(feat_proba.sort_values('Diff (Attrited-Existing)', ascending=False
                             ).round(3).head(8).to_string())

```

Questions d'analyse

10. BernoulliNB sur features binaires donne-t-il un F1 comparable à GaussianNB sur features continues ? Pourquoi la binarisation perd-elle de l'information ?
11. Regardez le tableau de probabilités $P(\text{feature}=1|\text{classe})$. Quelle feature binaire est la plus discriminante (plus grande différence Attrited – Existing) ?
12. Si vous augmentez α de 1 à 10 dans BernoulliNB, les probabilités convergent-elles vers 0.5 ? Pourquoi est-ce le cas avec un lissage fort ?

Q 2.3

MultinomialNB — détection de spam SMS

Consigne

Appliquez MultinomialNB sur le dataset SMS Spam Collection. Utilisez le pipeline classique **CountVectorizer** → **TF-IDF** → **MultinomialNB** pour la classification de texte.

```

# Dataset SMS Spam Collection (télécharger depuis UCI ou kaggle)
# Alternative : créer un mini-dataset pour la démonstration
sms_data = [
    ('ham', 'I will call you later tonight'),
    ('ham', 'Are you coming to the meeting tomorrow'),
    ('ham', 'Great to hear from you, see you soon'),
    ('ham', 'Can we reschedule for Friday afternoon'),
    ('ham', 'Thanks for the reminder about the project'),
    ('spam', 'WINNER You have won a FREE prize call now'),
    ('spam', 'Claim your FREE cash reward click here'),
    ('spam', 'Congratulations you have been selected FREE'),

```

```

    ('spam', 'URGENT reply now to claim your prize'),
    ('spam', 'FREE offer limited time only click now'),
]

# Charger le vrai dataset SMS si disponible
try:
    sms_df = pd.read_csv('SMSSpamCollection', sep='\t',
                        names=['label', 'text'])
    print(f'Vrai dataset SMS chargé : {sms_df.shape}')
except FileNotFoundError:
    sms_df = pd.DataFrame(sms_data, columns=['label', 'text'])
    print('Utilisation du mini-dataset de démonstration')

sms_df['label_num'] = (sms_df['label']=='spam').astype(int)
print(sms_df['label'].value_counts())

X_sms_tr, X_sms_te, y_sms_tr, y_sms_te = train_test_split(
    sms_df['text'], sms_df['label_num'],
    test_size=0.2, random_state=42, stratify=sms_df['label_num'])

# Pipeline : TF-IDF + MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer

# Option 1 : Bag of Words (comptages bruts)
pipe_bow = Pipeline([
    ('vect', CountVectorizer(stop_words='english', max_features=5000)),
    ('clf', MultinomialNB(alpha=1.0))
])

# Option 2 : TF-IDF
pipe_tfidf = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', max_features=5000)),
    ('clf', MultinomialNB(alpha=1.0))
])

for name, pipe in [('BoW+MNB', pipe_bow), ('TF-IDF+MNB', pipe_tfidf)]:
    pipe.fit(X_sms_tr, y_sms_tr)
    y_pred = pipe.predict(X_sms_te)
    print(f'\n=== {name} ===')
    print(classification_report(y_sms_te, y_pred,
                                target_names=['Ham', 'Spam']))

# Top mots discriminants pour SPAM
vect = pipe_bow.named_steps['vect']
clf_bow = pipe_bow.named_steps['clf']
feature_names = np.array(vect.get_feature_names_out())
log_proba_diff = clf_bow.feature_log_prob_[1] - clf_bow.feature_log_prob_[0]
top_spam_words = feature_names[np.argsort(log_proba_diff)[-15:]]
print(f'\nTop 15 mots les plus spam : {top_spam_words[::-1]}')

```

📌 MultinomialNB et valeurs négatives

MultinomialNB requiert des **features non-négatives**. Si vous essayez de l'appliquer sur BankChurners avec des features standardisées (valeurs < 0), vous obtiendrez une erreur. Toujours utiliser CountVectorizer ou MinMaxScaler(feature_range=(0,...)) avant MultinomialNB.

Questions d'analyse

13. Quels mots sont les plus discriminants pour la classe SPAM ? Ces mots correspondent-ils à vos intuitions sur les SMS indésirables ?
14. TF-IDF + MultinomialNB donne-t-il de meilleurs résultats que BoW + MultinomialNB ? Expliquez pourquoi TF-IDF peut améliorer les performances.
15. Pourquoi MultinomialNB est-il inadapté directement sur BankChurners (sans transformation préalable) ?

Partie 3 — Analyse de l'Hypothèse d'Indépendance

Durée estimée : 20 minutes | Quantifier l'impact de la violation de l'hypothèse centrale de Naive Bayes.

Q 3.1

Mesurer la violation d'indépendance — matrice de corrélation

Consigne

Calculez la matrice de corrélation des features de BankChurners et identifiez les paires fortement corrélées. Évaluez si la violation d'indépendance nuit significativement à GaussianNB.

```
# Violation de l'hypothèse d'indépendance
corr_matrix = X_tr_b.corr()

# Identifier les fortes corrélations
strong_corr = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        r = corr_matrix.iloc[i, j]
        if abs(r) > 0.5:
            strong_corr.append({
                'Feature 1': corr_matrix.columns[i],
                'Feature 2': corr_matrix.columns[j],
                'Corrélation': round(r, 3)
            })

corr_df = pd.DataFrame(strong_corr).sort_values('Corrélation',
                                                key=abs, ascending=False)

print('=== PAIRES CORRÉLÉES (|r| > 0.5) ===')
print(corr_df.to_string(index=False))

# Impact sur GaussianNB : comparer avec features décorrélées
# Sélectionner uniquement les features peu corrélées
from sklearn.feature_selection import SelectKBest, mutual_info_classif

# Sélection par information mutuelle (pas de lien linéaire)
selector = SelectKBest(mutual_info_classif, k=8)
selector.fit(X_tr_b, y_tr_b)
X_top8 = selector.transform(X_tr_b)
X_top8_te = selector.transform(X_te_b)

gnb_top8 = GaussianNB()
gnb_top8.fit(X_top8, y_tr_b)
f1_all = f1_score(y_te_b, gnb.predict(X_te_b))
f1_top8 = f1_score(y_te_b, gnb_top8.predict(X_top8_te))

print(f'\nGaussianNB (toutes features) : F1 = {f1_all:.4f}')
print(f'GaussianNB (top 8 MI) : F1 = {f1_top8:.4f}')
```

🔗 Rappel de cours — Hypothèse d'indépendance — Pourquoi ça marche quand même ?

- La violation d'indépendance affecte les **probabilités absolues** mais pas nécessairement la **décision finale** (argmax)
- Même si $P(C|X)$ est mal calibré, la classe prédite peut être correcte si les erreurs affectent toutes les classes de manière similaire
- **Domaine où NB excelle** : classification de texte (spam) — les mots sont souvent cooccurents mais NB reste robuste
- **Domaine où NB souffre** : features très corrélées (ex : Credit_Limit et Avg_Open_To_Buy sont presque la même chose)

Questions d'analyse

- Combien de paires de features ont $|r| > 0.5$ sur BankChurners ? Ces features corrélées violent-elles l'hypothèse d'indépendance de NB ?
- L'utilisation des 8 features les moins redondantes (top mutual information) améliore-t-elle les performances de GaussianNB ? Qu'est-ce que cela indique sur le rôle de la redondance ?
- Réflexion** : si Credit_Limit et Avg_Open_To_Buy sont corrélées à 0.99, inclure les deux dans NB revient à compter cette information **deux fois**. Comment le lissage de Laplace aide-t-il (ou ne l'aide-t-il pas) ?

Q 3.2

Calibration des probabilités — NB vs Régression Logistique

Consigne

Comparez la calibration des probabilités de GaussianNB et de la Régression Logistique. Un modèle bien calibré dit $P=0.7 \rightarrow 70\%$ de vrais positifs dans ce groupe.

```
from sklearn.calibration import CalibrationDisplay
from sklearn.linear_model import LogisticRegression

# Modèles à comparer
models_cal = {
    'GaussianNB': GaussianNB(),
    'Régression Logistique': Pipeline([
        ('sc', StandardScaler()),
        ('lr', LogisticRegression(max_iter=1000, random_state=42))
    ]),
}

fig, ax = plt.subplots(figsize=(9, 7))
ax.plot([0,1], [0,1], 'k--', lw=1, label='Calibration parfaite')

for name, model in models_cal.items():
    model.fit(X_tr_b, y_tr_b)
    CalibrationDisplay.from_estimator(
        model, X_te_b, y_te_b,
        n_bins=10, ax=ax, name=name
    )
```

```
ax.set_title('Courbes de Calibration — NB vs Logistique',
             fontweight='bold', fontsize=13)
ax.set_xlabel('Probabilité prédite moyenne')
ax.set_ylabel('Fraction de positifs (réelle)')
ax.legend() ; plt.tight_layout() ; plt.show()

# Brier score (plus bas = mieux calibré)
from sklearn.metrics import brier_score_loss
for name, model in models_cal.items():
    prob = model.predict_proba(X_te_b)[: ,1]
    brier = brier_score_loss(y_te_b, prob)
    print(f'{name:30s} : Brier = {brier:.4f}')
```

📌 Brier Score — Métriques de Calibration

- **Brier Score** = MSE des probabilités prédites : $\sum (p_i - y_i)^2 / n$ — entre 0 (parfait) et 1 (nul)
- Un modèle peut avoir un bon F1 mais être **mal calibré** (ses probabilités ne reflètent pas la réalité)
- **GaussianNB** est souvent over-confident (probabilités proches de 0 ou 1) car les hypothèses sont fortes
- **Régression Logistique** est généralement mieux calibrée pour les problèmes binaires
- La calibration est cruciale si vous utilisez les probabilités pour **prendre des décisions business** (ex : contacter le top 20% à risque)

Questions d'analyse

19. La courbe de calibration de GaussianNB est-elle proche de la diagonale parfaite ? Est-il over-confident ou under-confident ?
20. La Régression Logistique est-elle mieux calibrée que GaussianNB ? Quel est le Brier Score de chaque modèle ?
21. **Décision métier** : si vous devez contacter les 500 clients les plus susceptibles de churner (top probabilités), quel modèle préférez-vous en termes de calibration ? Pourquoi ?

Partie 4 — Benchmark Complet et Positionnement

Durée estimée : 20 minutes | Situer Naive Bayes dans le paysage des algorithmes étudiés jusqu'ici.

Q 4.1

Benchmark 5 modèles — TD 4 à 7

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
import time

all_models = {
    'Naive Bayes (Gaussian)': GaussianNB(),
    'Régression Logistique': Pipeline([
        ('sc', StandardScaler()),
        ('lr', LogisticRegression(C=1, max_iter=1000, random_state=42))
    ]),
    'Arbre de Décision': DecisionTreeClassifier(max_depth=8, random_state=42),
    'KNN (K=7)': Pipeline([
        ('sc', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=7))
    ]),
    'SVM RBF': Pipeline([
        ('sc', StandardScaler()),
        ('svc', SVC(kernel='rbf', C=1, probability=True, random_state=42))
    ]),
}

print(f"{'Modèle':30s} {'F1 CV':>10s} {'±':>8s} {'F1 Test':>10s} {'AUC':>8s} {'Temps (s)':>10s}")
print('-' * 82)

for name, model in all_models.items():
    t0 = time.time()
    cv_f1 = cross_val_score(model, X_tr_b, y_tr_b,
                            cv=5, scoring='f1', n_jobs=-1)
    model.fit(X_tr_b, y_tr_b)
    t_train = time.time() - t0
    y_p = model.predict(X_te_b)
    f1_t = f1_score(y_te_b, y_p)
    try:
        proba = model.predict_proba(X_te_b)[:,1]
        auc = roc_auc_score(y_te_b, proba)
    except:
        auc = float('nan')
    print(f'{name:30s} {cv_f1.mean():>10.4f} {cv_f1.std():>8.4f} '
          f'{f1_t:>10.4f} {auc:>8.4f} {t_train:>10.2f}')
```

Questions d'analyse

22. Naive Bayes est-il le modèle le plus rapide à entraîner ? Comparez son temps d'entraînement avec SVM et KNN.

23. Malgré son hypothèse forte violée, GaussianNB est-il compétitif en termes de F1 ? À quel prix (AUC, calibration) ?
24. Dans quels contextes recommanderiez-vous Naive Bayes malgré des performances parfois inférieures ? Pensez à : données en streaming, très grande dimension, interprétabilité.

Tableau de positionnement — Quand utiliser Naive Bayes ?

Contexte	NB recommandé ?	Alternative si non
Classification de texte (spam, sentiment)	✓ MultinomialNB — très bon	SVM linéaire, BERT
Features numériques corrélées	✗ Violation forte d'indépendance	Régression Logistique
Dataset très large (>1M exemples)	✓ GaussianNB $O(nd)$ — très rapide	SGDClassifier
Mise à jour en ligne (streaming)	✓ <code>partial_fit()</code> natif	River (ML en ligne)
Interprétabilité des probabilités	⚠ Calibration souvent mauvaise	Régression Logistique calibrée
Baseline rapide à établir	✓ Excellent 1er benchmark	—
Peu de données (<100 exemples)	✓ Bonne généralisation avec peu	Régression Logistique + L2

Section Bonus — Pour aller plus loin

Ces exercices explorent l'apprentissage en ligne et la calibration avancée.

Bonus B1 — Apprentissage en ligne avec `partial_fit()`

GaussianNB supporte `partial_fit()` — il peut apprendre progressivement sans stocker toutes les données.

```
# Simulation d'apprentissage en ligne (streaming par mini-batches)
gnb_online = GaussianNB()
batch_size = 500
f1_history = []

# Entraîner progressivement
X_arr = X_tr_b.values
y_arr = y_tr_b.values

for i in range(0, len(X_arr), batch_size):
    X_batch = X_arr[i:i+batch_size]
    y_batch = y_arr[i:i+batch_size]
    gnb_online.partial_fit(X_batch, y_batch, classes=[0, 1])
    # Évaluer après chaque batch
    f1 = f1_score(y_te_b, gnb_online.predict(X_te_b.values))
    f1_history.append(f1)

plt.figure(figsize=(10, 5))
plt.plot(range(1, len(f1_history)+1), f1_history,
         'o-', color='#1A7A6E', lw=2)
plt.axhline(f1_score(y_te_b, gnb.predict(X_te_b)),
            color='#9C3D38', ls='--', label='GaussianNB batch complet')
plt.xlabel(f'Nombre de mini-batches ({batch_size} exemples/batch)')
plt.ylabel('F1-Score test') ; plt.legend()
plt.title('Convergence de GaussianNB en apprentissage online',
         fontweight='bold')
plt.show()
```

25. Le F1-score converge-t-il vers la valeur du batch complet après suffisamment de mini-batches ?
Combien de batches sont nécessaires ?

26. Quel algorithme autre que GaussianNB supporte `partial_fit()` dans sklearn ? (hint : cherchez dans la documentation)

Bonus B2 — Calibration de Naive Bayes avec `CalibratedClassifierCV`

```
from sklearn.calibration import CalibratedClassifierCV

# Calibrer GaussianNB avec isotonic regression
gnb_calibrated = CalibratedClassifierCV(
    GaussianNB(),
    method='isotonic', # 'sigmoid' (Platt) ou 'isotonic'
    cv=5
)
gnb_calibrated.fit(X_tr_b, y_tr_b)
```

```
# Comparer calibration avant/après
from sklearn.metrics import brier_score_loss
prob_gnb = gnb.predict_proba(X_te_b)[:,-1]
prob_cal = gnb_calibrated.predict_proba(X_te_b)[:,-1]

print(f'Brier score GaussianNB brut      : {brier_score_loss(y_te_b, prob_gnb):.4f}')
print(f'Brier score GaussianNB calibré   : {brier_score_loss(y_te_b,
prob_cal):.4f}')
print(f'F1 GaussianNB brut               : {f1_score(y_te_b,
gnb.predict(X_te_b)):.4f}')
print(f'F1 GaussianNB calibré           : {f1_score(y_te_b,
gnb_calibrated.predict(X_te_b)):.4f}')
```

Livrable attendu

✓ Ce que vous devez remettre

- Notebook **TD7_NomPrenom.ipynb** — toutes les cellules exécutées avec outputs visibles
- **Calcul manuel** Naive Bayes sur le dataset météo avec vérification Python
- **Démonstration** du problème de probabilité nulle et solution avec lissage de Laplace
- **Comparaison GaussianNB, BernoulliNB, MultinomialNB** — tableaux et interprétations
- **Analyse de calibration** (courbes + Brier Score) comparant NB et Régression Logistique
- **Benchmark final** 5 modèles avec temps d'entraînement inclus
- Une cellule Markdown de **conclusion** (~150 mots) : forces et faiblesses de Naive Bayes, cas d'usage recommandés
- Délai : **avant le TD 8**, dépôt sur la plateforme pédagogique

Critères d'évaluation

Critère	Coefficient	Description
Calcul manuel Bayes + lissage Laplace	25%	Exactitude et explication pas à pas
Comparaison des 3 variantes NB	25%	Choix justifié, analyse des paramètres
Analyse de l'hypothèse d'indépendance	20%	Corrélations + impact sur les perf.
Calibration et benchmark final	20%	Courbes, Brier Score, positionnement
Section Bonus (optionnel)	+ 10%	partial_fit ou CalibratedClassifierCV

Ressources et Documentation

- **sklearn — Naive Bayes** : https://scikit-learn.org/stable/modules/naive_bayes.html
- **sklearn — Calibration** : <https://scikit-learn.org/stable/modules/calibration.html>
- **Dataset SMS Spam** : <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
- **sklearn — CountVectorizer** : https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
- **Article fondateur : Rish (2001)** — An empirical study of the naive Bayes classifier

Paramètres clés des variantes Naive Bayes

Variante	Paramètre clé	Impact
GaussianNB	var_smoothing (1e-9)	Ajoute petite variance pour stabilité numérique

Variante	Paramètre clé	Impact
BernoulliNB	alpha (1.0)	Lissage de Laplace : 0=aucun, 1=standard
MultinomialNB	alpha (1.0)	Lissage de Laplace pour les fréquences de mots
Tous	class_prior (None)	Probabilités a priori — None=calculé des données
GaussianNB	(incremental)	partial_fit() pour l'apprentissage en ligne